Forecasting versus Reactive Cloud Auto-Tiering: A Pre-Registered, Fee-Accurate Test

Summit Khatiwada

Independent Researcher, Kathmandu, Nepal

Version 1.0, October 2026. Code, data records and every result:
https://github.com/summitkhatiwada22/seasonal-tiering-poc

Abstract

Cloud archives decide where data lives by looking backwards: S3 Intelligent-Tiering, Google Autoclass and Azure smart tier move a file down after it has gone unread for some time, and move it back when it is read. We asked whether looking forwards does better: whether forecasting when archived files will be read lets a storage plan keep them in the cheapest deep tiers and bring them back before anyone asks. We built a fee-accurate simulator that bills every file every day under the providers' published rules. It matched 24 hand calculations and the AWS Pricing Calculator, and caught 48 deliberately planted bugs. We drove it with public demand data (Wikipedia page views, npm downloads and Seattle library checkouts) mapped to archive reads. All methods were fixed before they were scored, in three steps: an early check on 2023, two rolling years (gate R1), and a sealed test on 2024 and 2025. The sealed test was pre-registered and run once, from a frozen git tag that the code required before it would open those years. Seasonal forecasting on its own never beat Intelligent-Tiering. A per-file choice among 161 simulated policies was 39% to 54% cheaper on Wikipedia and npm in the sealed years when up to 1% of reads may wait. It saved by leaving a few large, rarely read files in deep archive, behind restores of up to 48 hours. It still failed the pre-registered test: in 2024, slightly more of its reads waited than the rival's (0.277% against 0.245%, and 0.118% against 0.089%). It lost on the library data in every year, and with no waiting allowed nothing beat managed tiering. A reference that knew each file's true read rate held the waiting line on every Wikipedia and npm line, so the per-file economics hold. What fails is predicting next year's reads from last year's.

Keywords: cloud storage, storage tiering, S3 Intelligent-Tiering, archival storage, demand forecasting, pre-registration, negative result.

1 Introduction

Storage is billed every month, and most stored data is rarely read. Cloud providers sell a ladder of storage classes: on AWS, keeping a gigabyte for a month costs 0.023 USD in S3 Standard and 0.00099 USD in Glacier Deep Archive, 23 times less [2]. The catch is in the fees and the delays. Deep tiers charge to read data back, bill a minimum storage period, and make a reader wait hours for a restore.

The providers' answer is managed, reactive tiering. S3 Intelligent-Tiering moves an object to cheaper tiers after 30, 90 or more days without access and back on the next read [1]. Google Autoclass and Azure smart tier work the same way [7, 13]. These services move data between their own tiers without retrieval, transition or early-deletion fees, but they cannot see the future: a file read once a year pays to sit in a warm tier for weeks before it moves down, and its next reader may wait.

The idea tested here is to look forwards. If an archive could forecast which files will be read, and when, it could keep them in the deepest tiers and restore them shortly before they are wanted. It could even use the slow bulk restores that are free or nearly free but take up to 48 hours [3], which a reactive service cannot wait for. Demand for many kinds of content is seasonal, so even a simple forecast ("the same weeks last year") might be enough.

Vendor claims of 50% to 80% savings from smarter tiering are common, and they rarely state the baseline. We found no study that compares a forecasting method against these managed services under their real fee rules. So we ran one, with three commitments made before any result was seen:

(1) **Fee-accurate costs.** Every number comes from one simulator that bills each file each day under the providers' published rules, including the managed services' own rules. The simulator was checked against hand calculations, the AWS Pricing Calculator and planted bugs before it was used (Section 4).

(2) **The strongest rival.** Ours is compared with the cheapest rival setting, chosen on the same past data, not with "keep everything hot".

(3) **Rules before data.** Each test's pass rule was committed to git before its data was scored. The final, sealed test was pre-registered [16] and run once from a frozen tag; the code refused to open the sealed years anywhere else (Section 6).

Contributions.

- A fee-accurate, per-file, day-by-day simulator of AWS, Google and Azure storage billing, including S3 Intelligent-Tiering, Autoclass and smart tier, with its validation record (Section 4).
- A pre-registered, git-enforced evaluation protocol for storage policies on public demand data (Section 6).
- A negative result: forecasting does not beat reactive auto-tiering on these data (Section 7).
- A diagnosis of why. The savings that do exist are concentrated in a few large, rarely read files. They are lost because a year-ahead choice cannot hold waiting at the rival's level, not because the economics are wrong (Section 8).
- All code, protocols, run outputs and the decision log, open for reuse [10].

2 Background: what archive storage costs

Table 1 lists the AWS classes we simulate. Three things make deep tiers harder to use than their storage price suggests. First, *retrieval*: every read from an archive class pays per gigabyte and per request, and Glacier Flexible Retrieval and Deep Archive need a restore before the read, which takes hours. Second, *minimum storage*: an object deleted or moved within 90 or 180 days is billed for the rest of that period. Third, *overheads*: archive classes add 40 KB of billed metadata per object, and the infrequent classes bill small objects as if they were 128 KB.

Managed tiering avoids all three costs for moves between its own tiers. Intelligent-Tiering charges a small monitoring fee per object instead, and its archive tiers make the next reader wait for a restore. Its rules are fixed and reactive. Any plan that beats it must save more on storage than it pays in fees, and must not make many more readers wait.

Table 1. The AWS us-east-1 price sheet used (price data of 2026-10-15) [1–3]. Restore times are the upper ends AWS states, used in the central setting.

Class	Storage (USD/GB-month)	Min. days	Retrieval (USD/GB)	Restore before read
S3 Standard	0.023	–	–	none
Standard-IA	0.0125	30	0.01	none
Glacier Instant Retrieval	0.004	90	0.03	none
Glacier Flexible Retrieval	0.0036	90	0.01 (bulk: 0)	5 h (bulk: 12 h)
Glacier Deep Archive	0.00099	180	0.02 (bulk: 0.0025)	12 h (bulk: 48 h)

S3 Intelligent-Tiering: 0.023 frequent; 0.0125 after 30 days unread; 0.004 after 90; optional archive tiers (0.0036 after 90+ days, 5 h restore; 0.00099 after 180+ days, 12 h). No retrieval or early-deletion fees. Monitoring 0.0025 USD per 1,000 objects a month; objects under 128 KB stay frequent and are not charged.

3 Related work

Cost-aware cloud storage placement. Choosing where to put data to minimise a cloud bill has a long history. FCFS combined several cloud storage services and resized them to cut file-system costs [18]. SPANStore placed replicas across providers' regions to meet latency goals at least cost [25]. SkyStore balances storage against cross-region egress with an adaptive time-to-live per replica [11]. Erradi and Mansouri give online algorithms for hot and cool tiers with a competitive ratio below two, and report 5% to 55% savings over keeping data hot on a Twitter workload [6]. SCOPe forecasts per-partition access and chooses tiers and compression together, reporting 50% to 83% savings on enterprise data lakes [14]. None of these compares against today's managed auto-tiering services with their actual fee rules, and none was evaluated under a pre-registered rule. Their baselines are typically fixed placements or keeping data hot. Our results suggest those are much weaker rivals than Intelligent-Tiering.

Cold storage systems. Inside large operators, warm and cold data have their own systems, such as Facebook's f4 [15] and Microsoft's Pelican [4]. They lower the cost of storing cold data. We take the providers' classes as given and ask how to use them.

Learning-augmented caching and decisions. Caching with predictions is close to our problem: an archive is a cache whose misses cost money and time. Learning-augmented online algorithms trade consistency, when predictions are good, against robustness, when they are bad [12, 23]. LRB learns to approximate Belady's optimal eviction for CDN caches [22]. "Predict, then optimise" argues that forecasts should be judged by the decisions they lead to, not by forecast error [5]. Our per-file chooser is a simple decision-focused rule of this kind. Our main finding, that it cannot bound its own waiting, is the robustness problem these papers study.

Forecasting and benchmarking practice. We use the seasonal-naive forecast ("the same days last year"), the standard baseline in forecasting practice [9]. On methodology, we follow calls for pre-registration [16] and for stating baselines, variability and conditions in systems benchmarking [8].

4 Data and simulator

4.1 Datasets

No public dataset records reads of an archive's files, so we use public demand series as stand-ins and map demand to reads (Table 2). They differ in how predictable they are (Figure 2). npm's

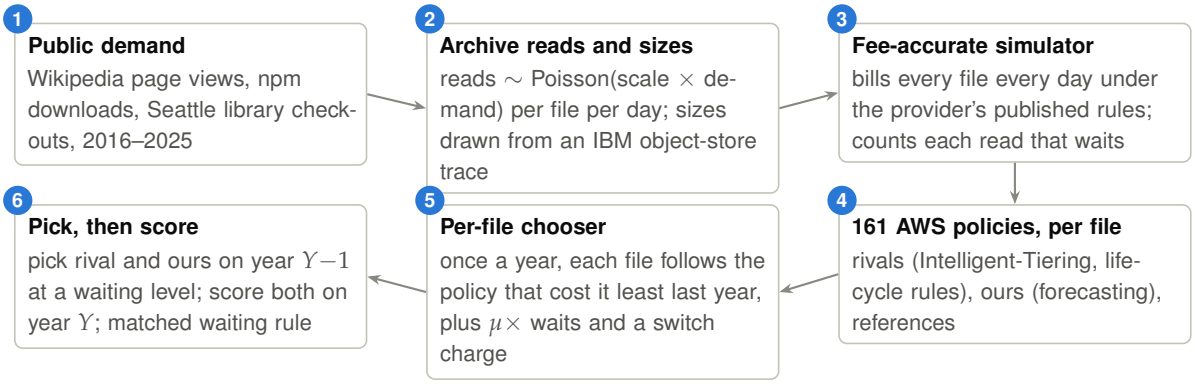


Figure 1. How every result is produced. Public demand becomes archive reads (1–2). The simulator bills each file under each of 161 policies (3–4). The per-file chooser combines them (5). The pick-then-score rule compares ours with the best rival on a year neither has seen (6).

Table 2. Datasets. Counts are for the sealed years 2024 and 2025.

Dataset	Source and sample	Files	Stored
D1 Wikipedia	Daily user page views [24] of 10,000 articles: the 5,000 most viewed and 5,000 at random	9,684–9,867	0.19 TB
D3 npm	Daily downloads [17] of 300 packages: the 100 most used and 200 at random from the next 4,900	295–298	4 GB
D4 Library	Monthly checkouts [20] of a fixed random sample of 10,000 titles, spread evenly over each month	10,000	0.21 TB

within-year profile repeats: it rises steadily through each year and drops sharply at the end-of-year holidays. Wikipedia's is nearly flat, with spikes. The library's has no stable shape, and two events broke it: the 2020 closures, and a ransomware attack on the Seattle Public Library in late May 2024 [19]. In the sample, checkouts fell from about 4,000 a month in January to April 2024 to 1,114 in May and 670 in June.

From demand to reads. Views and downloads are mostly served by caches and mirrors, not by an archive. We therefore draw each file's daily reads as Poisson$(s \cdot d)$, where d is its demand that day and s is one scale per dataset (one per cohort for Wikipedia). The central scale makes the median file read about 18 times a year, set on 2016–2022 data. The low and high settings use 10 times less and 10 times more. Draws come from fixed random streams, so every run is reproducible. A file exists from its first day of demand.

File sizes. None of the sources has file sizes. We draw them from the IBM object-store trace in the SNIA IOTTA repository [21] (central setting), with Wikimedia media sizes as the low setting and the trace's upper half as the high one. In the central setting the median file is 56–68 KB, the 99th percentile about 0.8 GB, and the largest about 1 GB.

Absolute bills are small, tens of dollars a year, because the samples are small. The comparisons are relative. Storage scales with bytes and request fees with files, so a larger archive with the same mix of sizes and reads would show the same percentages.

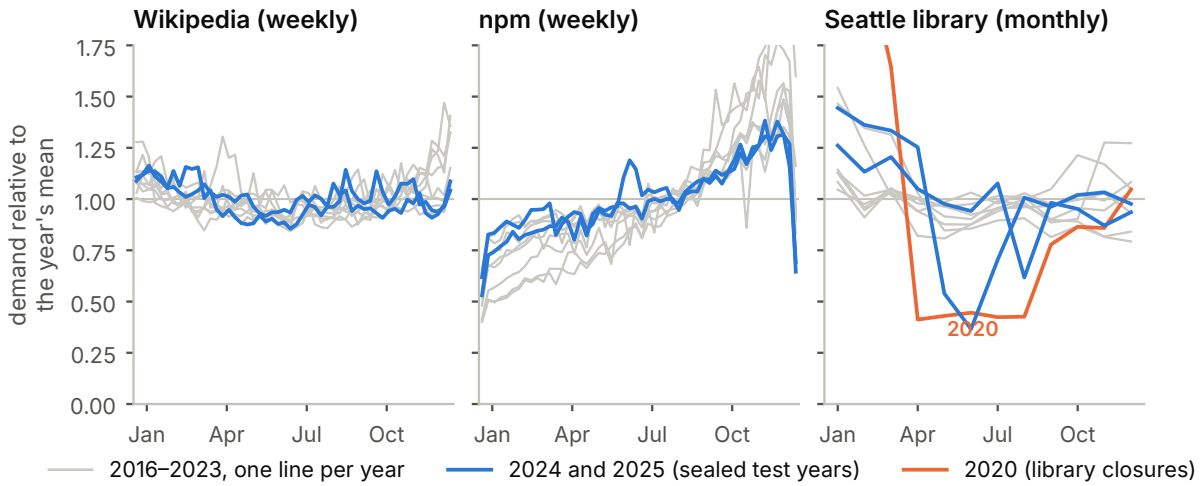


Figure 2. Within-year demand of the average file, one line per year. Each file's demand is divided by its own mean for that year, then averaged over files. The sealed years (blue) look like earlier years on Wikipedia and npm. On the library data, 2020 (orange) and 2024 (the dip in May and June, during a ransomware attack on the library) break the pattern.

4.2 Simulator

The simulator takes, for each file, its size, its creation day and its reads per day, and a plan: which class the file is in each day, plus any restores ordered ahead of time. It returns a bill in ten categories: storage, archive overhead, temporary restored copies, early deletion, transitions, retrieval, restore requests, read requests, uploads and monitoring. It also returns every read that found its file frozen, with how long it waited. Rules are taken from the providers' documentation and quoted with their sources in the repository. They include minimum billable sizes, per-object archive overhead, restores and their temporary copies, and the managed services' own state machines.

Validation (gate G2). Before any real data was used, the simulator matched 24 of 24 hand calculations to a relative error below 10^{-9} and 4 of 4 AWS Pricing Calculator readings to the cent. It caught 22 of 22 deliberately planted bugs. By the end of the project, the planted-bug check covered 48 bugs across the simulator, the policies and the protocol code, and caught all of them.

5 Methods compared

All methods run on every file. Files never interact in the simulator, so a plan that gives each file its own policy costs exactly the sum of each file's cost under its policy. We simulate 161 AWS policies.

Rivals (19). S3 Intelligent-Tiering in six settings (with and without its archive tiers); age-based lifecycle rules that move every file to one class 30 or 90 days after upload, for every class; and the archive rules again with bulk restores on demand. As on AWS, files under 128 KB never leave Standard under a lifecycle rule.

Ours (91, plus the chooser). Every method of ours forecasts with last year's reads.

- *Seasonal-naive rule:* a file is warm if it was read in the same days last year (364 days back, same weekday) or recently. Otherwise it sits in a cold class, or stays archived with restored

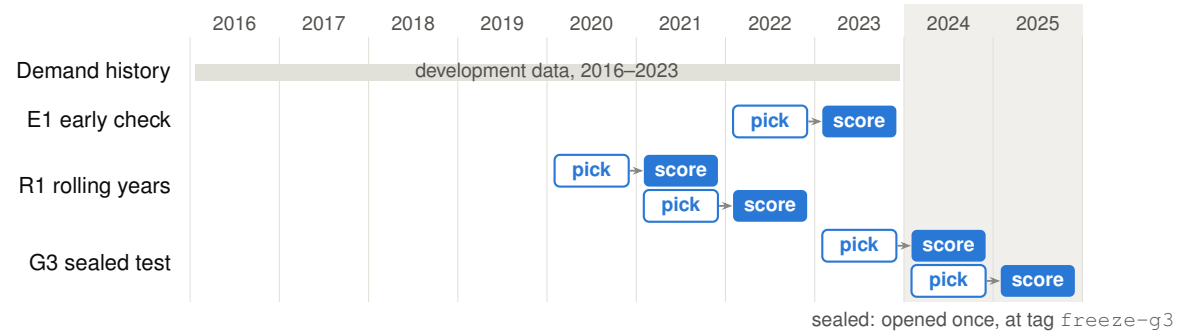


Figure 3. Which years each test used. Every method was picked on one year and scored on the next. The data loaders refused the sealed years 2024 and 2025; they were opened once, by code at the freeze tag.

copies ordered ahead of the forecast reads, at standard or bulk speed.

- *Fee-aware planner:* each day, puts each file in the class with the lowest expected cost over the next 30, 90 or 180 days, counting every fee.
- *Intelligent-Tiering with forecast restores:* uses Intelligent-Tiering's archive tiers, and restores a file a set number of days before last year's reads say it will be wanted.
- *Per-file chooser:* once a year, each file follows the policy (out of all 161, rivals included) with the lowest score

$$\text{score}(p,i) = \text{cost}_{Y-1}(p,i) + \mu \cdot \text{waits}_{Y-1}(p,i) + \text{switch}(i) \cdot [p \neq p_{\text{now}}(i)],$$

where cost and waits are file i's bill and waited reads under policy p last year. The switch charge is an upper bound on the one-off cost of moving the file (the dearest retrieval, request and early-deletion fees on the sheet). The file keeps its current policy unless the best score is lower by more than a margin m. Files without history stay on plain Intelligent-Tiering. There are 21 settings: $\mu \in \{0, 0.001, 0.01, 0.1, 1, 10\}$ USD per waited read, $m \in \{0, 0.1, 0.25\}$, and an "online" menu without policies under which reads can wait.

References (51, plus the ceiling). Each of our rules has a *reactive twin* with the forecast switched off, so a gain can be traced to the forecast rather than to reacting. The *forecastable ceiling* makes the chooser's choice on an independent random draw of the scored year's reads from the same expected rates. It knows each file's true read rate, but not the draw that is scored. It shows how much a perfect forecast of rates could earn.

6 Protocol

Pick, then score. A test is a set of *lines*: a dataset, a waiting level, and a pair of years (pick on $Y-1$, score on Y). There are two waiting levels: L0, where no read may wait, and L1, where at most 1% of reads may wait. On the pick year we choose the cheapest rival that meets the level. We then choose the cheapest of ours that waits no more than that rival. Both are scored on year Y. The *gap* is $1 - \text{cost}_{\text{ours}}/\text{cost}_{\text{rival}}$. Under the deciding *matched* rule, a line passes if the gap is at least 5% and ours again waits on no more reads than the rival in year Y. A second rule, reported only, gives ours the full 1% allowance.

Three tests (Figure 3). *E1*, an early futility check, picked on 2022 and scored 2023 once. *R1*, the gate for the per-file methods, used the rolling pairs 2020→2021 and 2021→2022. *G3*, the sealed

Table 3. Pre-registered hypotheses for the sealed test, and their outcome.

	Passes if (sealed years, AWS, matched rule)	Result
H1 Cost	At one level, both pairs: gap $\geq 5\%$, ours waits no more than the rival, 95% paired-bootstrap interval of the gap above zero (2,000 resamples over files). On at least 2 of the 3 datasets	**fail** on all three
H2 Waiting	At L1, both pairs: ours costs no more, and its share of waited reads is at most half the rival's. On 2 of 3	**fail** on all three
H3 Headroom	Ours' gap divided by the ceiling's gap (reported)	0.94 and 0.99 (Wikipedia); 0.76 and 1.99 (npm)
H4 Model needed	Ours against the seasonal-naive family alone (reported)	The seasonal-naive pick costs 0.6% to 29% more than the rival on all 12 lines
H5 Robust	The passing hypothesis holds in at least 8 of 10 scenarios	**fail**: nothing passed in base. H1 held in 2 of 10, H2 in 1
H6 Compute	Compute below 5% of the saving (0.04 USD per core-hour)	0.01% to 0.13%; not scored, since no dataset passed
H7 Simulator	Gate G2	**pass**
H9 Reproducible	A fresh clone at the tag reproduces every base line within 0.05%	**pass**: every line matched; run files byte-identical

test, used 2023→2024 and 2024→2025. The order of work is visible in git: E1's picks were committed before 2023 was run (d1cf04d), R1's protocol before any real-data run (0bde4de), and G3's pre-registration in the commit the freeze tag points to (cc4b852).

The sealed years. The data loaders refused any file from 2024 or later by its folder name, and no analysis opened those years before the sealed test. For G3 the code opened them only when the tag `freeze-g3` existed on GitHub and locally, pointed at the running commit, and no tracked file had changed. The author created the tag on GitHub before the sealed files were copied in. Their hashes and modification times, all from 2026-10-29, are recorded with the results.

Pre-registered hypotheses. Table 3 lists them. The proof of concept passes if (H1 or H2) and H5, H6, H7 and H9 all hold. The kill rule, also written in advance: if H1 and H2 both fail, the product thesis stops and the work becomes a paper.

Deviations. Three, all recorded before or as they happened. (1) Gate R1 failed (Section 7.2). By its rule the project should have stopped there. The author overrode it as a documented deviation: the methods were frozen unchanged and the sealed years opened once. R1 is reported as failed. (2) The library sample for 2024 and 2025 was extracted after the freeze, with the frozen extraction code. The same code first reproduced the pre-freeze 2023 sample exactly. (3) The workspace restarted during the ninth of ten scenarios. The run resumed from a checkout of the tag; the runs recomputed after the restart were byte-identical to those before it.

7 Results

7.1 Early check (E1, 2023)

E1 asked whether the simplest forecast showed any sign worth chasing. It passed by its own rule, but only against weak rivals. Every passing line was on Azure, against last-access rules

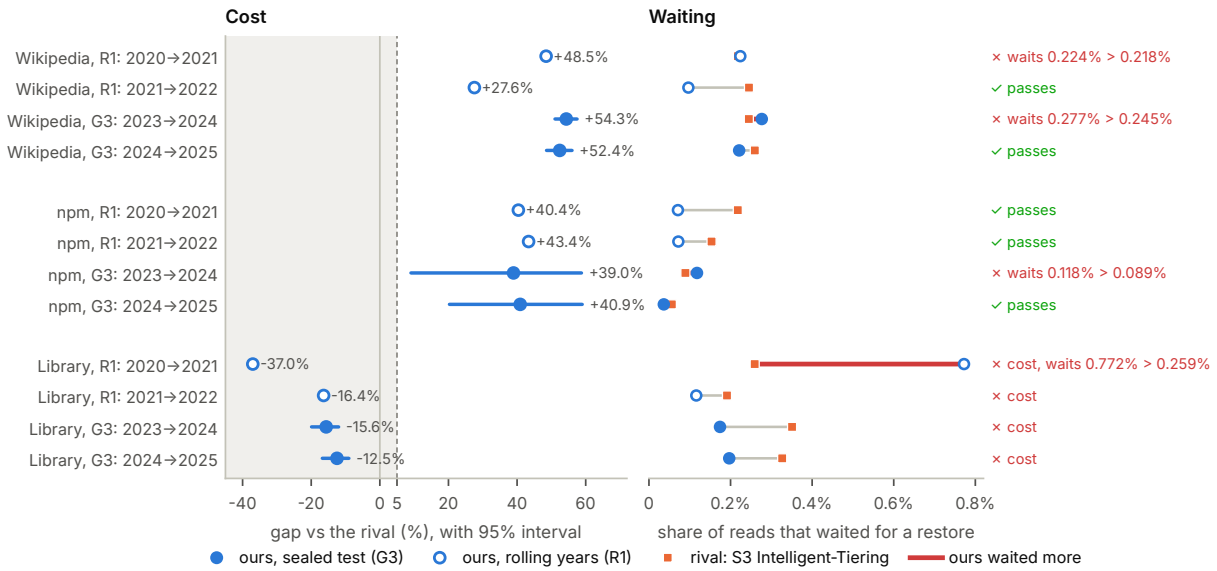


Figure 4. Every rolling (R1, open) and sealed (G3, filled) line at L1 under the matched rule. Left: how much cheaper ours was than the rival (S3 Intelligent-Tiering with its archive tiers), with the 95% paired-bootstrap interval for the sealed years. Right: the share of reads that waited, ours against the rival. A line passes only if ours is at least 5% cheaper and waits on no more reads. Wikipedia and npm failed in 2024 on waiting alone.

on a locally redundant sheet where smart tier is not offered (Wikipedia +9.1%, npm +23.1%, library +26.1%). Against Intelligent-Tiering and Autoclass, the seasonal-naive rule never passed: it lost by up to 69%, or was within half a percent. A perfect forecast of the scored year beat Intelligent-Tiering by 4% to 27% at L0, but came within −7% to +6% of Autoclass. We therefore dropped Google and built the per-file methods for AWS.

7.2 Rolling years (gate R1)

R1 failed narrowly. The chooser was 27.6% and 48.5% cheaper than Intelligent-Tiering on Wikipedia, and 40.4% and 43.4% on npm, at L1. It passed both rolling lines on npm, but on Wikipedia in 2021 0.224% of its reads waited against the rival's 0.218%. On the library data it lost, after the 2020 closures broke the link between one year and the next (Figure 4, open markers).

7.3 The sealed test (G3, 2024–2025)

H1 and H2 failed on every dataset (Table 3). Figure 4 shows why. On Wikipedia and npm, the chooser was 52% to 54% and 39% to 41% cheaper than the rival, with 95% intervals above zero. In 2024, though, slightly more of its reads waited than the rival's: 0.277% against 0.245% on Wikipedia, and 0.118% against 0.089% on npm. Each of those datasets failed one of its two sealed lines, and H1 needs both. On the library data, the cheapest of our policies that met the waiting rule on the pick year was Intelligent-Tiering with forecast restores. It lost by 15.6% and 12.5%. At L0, ours either equalled Intelligent-Tiering or, on npm in 2024, saved money only by letting a few reads wait. No L0 line passed.

Waiting in hours. The matched rule counts reads, not hours. Even on lines where ours waited on fewer reads, those reads waited longer: up to 48 hours (bulk restores from Deep Archive) against the rival's 12. In total, ours waited 2.9 to 4.3 times the rival's hours on the Wikipedia and

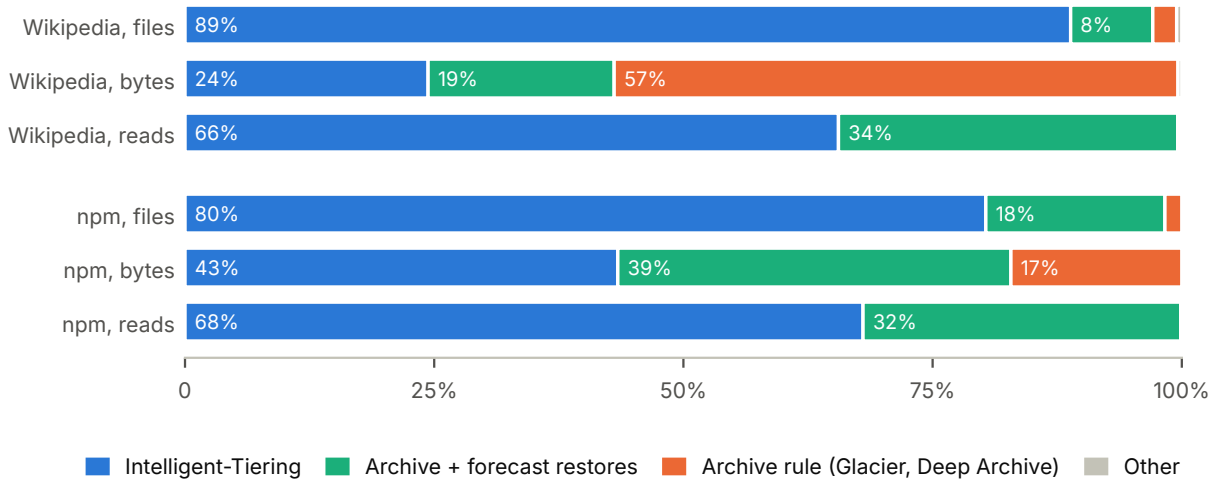


Figure 5. What the chooser did with the files in its 2024 sealed picks: the share of files, bytes and reads under each kind of policy. On Wikipedia, 2.4% of the files (median 318 MB) went to plain archive rules. They hold 57% of the bytes and receive 0.25% of the reads.

npm lines. With the reported-only equal allowance, the gaps are similar (+54.3% and +52.7% on Wikipedia, +20.9% and +49.3% on npm), but ours then waits on up to five times the rival's share of reads, and for up to 7.9 times its hours (Table 4 in the appendix).

Seasonality alone (H4). Picked by the same rule, the seasonal-naive family cost 0.6% to 29% more than the rival on all twelve base lines. Applied to every file, the forecast loses. The per-file chooser does use seasonal policies, but only for a minority of files (Section 8).

Azure. On Azure's zone-redundant sheet, where smart tier is offered and there is no archive tier, no read ever waits. There the chooser only ties smart tier: it is 0.02% to 0.10% cheaper in the sealed years.

Reproduction (H9). A fresh clone of the tag re-simulated and re-scored the base scenario. Every rival and ours cost matched, and the run files compared were byte-identical.

8 Why it fails

8.1 Where the money is

Figure 5 shows how the chooser made its saving. It left 80% to 89% of files on Intelligent-Tiering. It moved 8% to 18% of files, mid-sized ones that are read regularly, to archive with forecast restores. It sent a few very large, rarely read files to plain Glacier or Deep Archive rules; on Wikipedia, most of them with bulk restores. On Wikipedia in 2024 that last group was 232 of 9,684 files, with a median size of 318 MB. It held 57% of the bytes and received 0.25% of the reads. Storage is billed by the byte, so these few files decide most of the storage bill.

So the saving is mostly per-file economics rather than seasonality. Most of the bytes go to plain archive rules that need no forecast at all; the forecasting policies help only for the minority of files they are given. A file that is large, rarely read and read by someone willing to wait is far cheaper in Deep Archive than in any tier Intelligent-Tiering will put it in by itself. The chooser finds these files from last year's bill. What it cannot do is promise that next year's readers of those files will be as few as last year's.

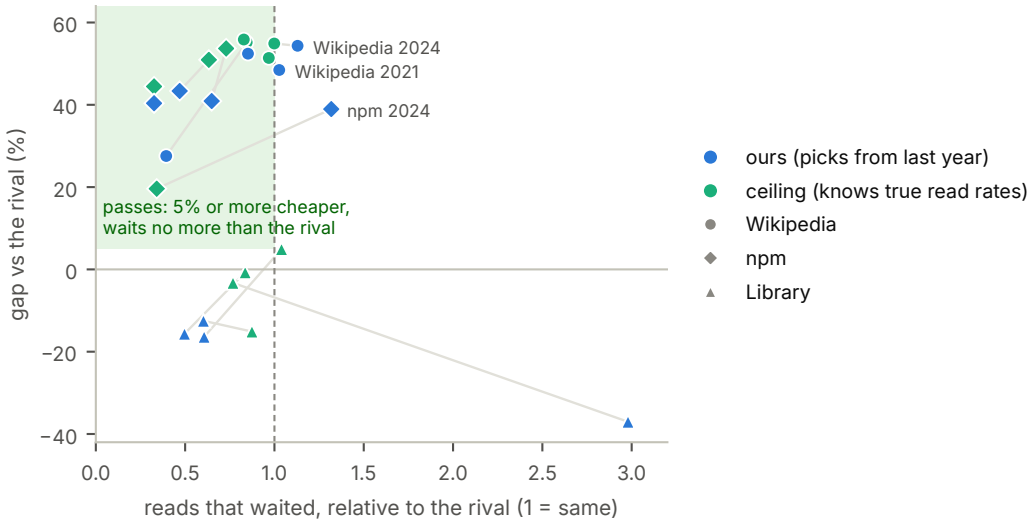


Figure 6. Each L1 line of R1 and G3, for ours (blue) and the forecastable ceiling (green), joined by a grey line. Horizontal axis: reads that waited, relative to the rival. Vertical axis: the gap. The ceiling, which knows each file's true read rate, lands in the passing region on all eight Wikipedia and npm lines. Ours crosses the waiting line three times.

8.2 The waiting line, and the ceiling

The chooser picks the setting whose waits on the pick year were no more than the rival's, and hopes the next year behaves the same. In one of the two sealed years it did not, on both datasets, and the same had happened once in R1. The forecastable ceiling makes the same kind of choice, but from each file's true expected read rate instead of last year's draw. It waited no more than the rival on all eight Wikipedia and npm lines of R1 and G3, while 20% to 56% cheaper (Figure 6). A good enough forecast of each file's *rate* would therefore have passed. Last year's reads were not one.

On the library data, even the ceiling's gap ranged from +4.9% to −15.1%. Knowing the rates does not help when the rates themselves shift. This happened twice: with the 2020 closures, and with the 2024 cyberattack, which fell inside the sealed test.

8.3 Robustness

Figure 7 repeats the sealed test in the ten pre-registered scenarios. These were storage prices and fees at 0.5 and 1.5 times, restore times at the low end of AWS's range, read scale and file sizes low and high. The pattern is the same throughout: large gaps on Wikipedia and npm, losses on the library, and passes that come and go. With storage at half price or fees at 1.5 times, the chosen settings waited no more than the rival in both sealed years, and H1 held. With small files the gaps shrank to about 3%, but ours almost never made a reader wait (at most 0.007% of reads, against 0.03% to 0.11% for the rival), so H2 held. Even if base had passed, H5 (8 of 10) would have failed.

9 Discussion

For practitioners. Two results are useful whatever one thinks of forecasting. First, S3 Intelligent-Tiering is a strong default: no rule of ours applied to all files beat it, and with no waiting allowed nothing did. Second, a large share of a bill can sit in a small number of large, rarely read files.

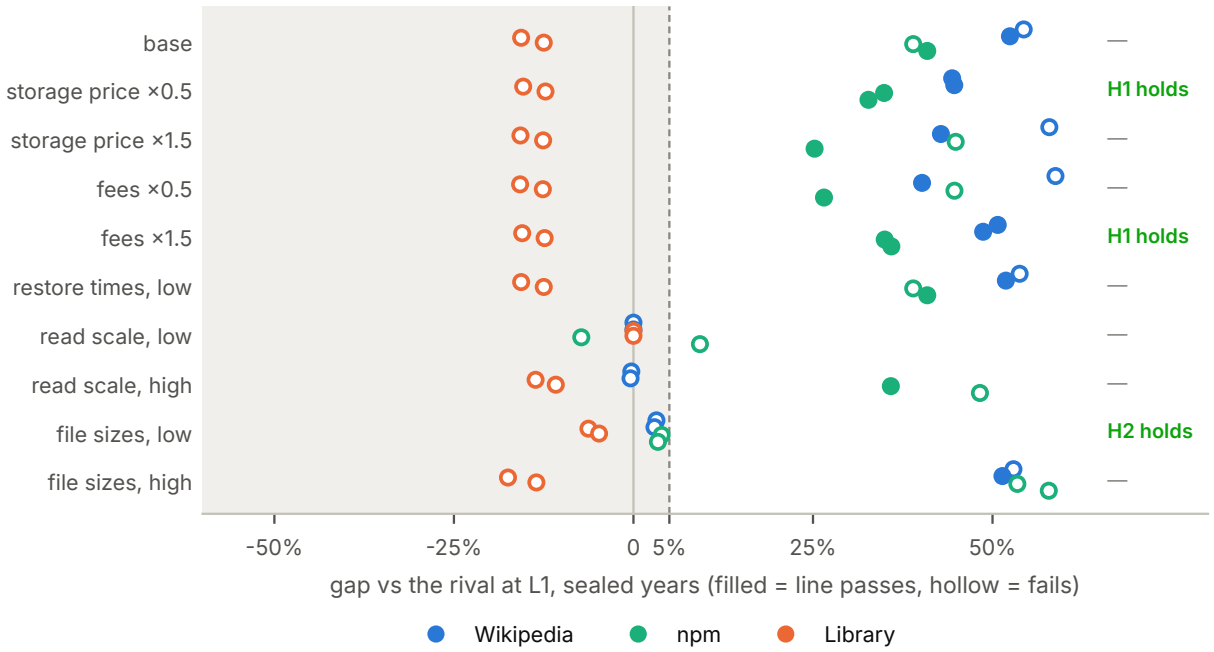


Figure 7. The four Wikipedia and npm sealed lines and two library lines at L1, in each of the ten pre-registered scenarios. Filled markers pass; hollow ones fail. The gap is large in most scenarios. Whether a line passes changes from scenario to scenario. H1 held in two scenarios and H2 in one; H5 needed eight.

For those files, a plain Deep Archive rule with bulk restores is far cheaper, if their readers can wait up to two days. Finding them needs only last year's per-file bill, which a fee-accurate simulation provides.

What would change the answer. (1) A controller that holds waiting under a budget *during* the year, rather than only at the yearly choice. This was planned as the next stage and not built after R1 failed. The ceiling suggests the target is reachable on stable data. The sealed years are spent, so testing it needs a new pre-registered test on later data. (2) A real archive's access log. Our reads are public demand scaled to archive levels; a real archive with large, seldom-read files would favour the per-file choice, and one with small or busy files would not. (3) A different notion of waiting. If waiting is judged in hours, ours looks worse. If a two-day restore is acceptable for the files in question, the saving stands without the waiting rule.

On pre-registration. The test failed by 0.03 percentage points of waited reads, on lines that were 39% to 54% cheaper. It is tempting to call that a pass in spirit. We do not, for two reasons. The same overshoot happened in R1, so it is a property of the method rather than bad luck. And a rule that can be relaxed after the data is seen gives no protection at all. The failure is what makes the rest of the evidence credible.

10 Threats to validity

- **No customer log.** Reads are mapped from public demand by a stated scale, and sizes are drawn from one trace. A real archive could favour either side.
- **Small sample.** Three datasets, two rolling and two sealed pairs. npm has fewer than 300 files, so its intervals are wide (lower ends +9.1% and +20.3%).
- **The waiting rule counts reads, not hours** (Section 7).

- **A deviation was taken.** The sealed test ran only because the author overrode a failed gate. It was pre-registered and frozen first, but it is not the path the stop rule prescribed.
- **The switch charge is an upper bound,** conservative for ours.
- **One region per cloud, list prices.** Discounts, other regions and later price changes are not modelled; the price scenarios cover part of this.
- **AWS only for the per-file methods.** Google was dropped after E1. Azure was tested only against smart tier on a sheet without an archive tier.
- **Single author.** The hand calculations have not yet been re-derived by a second person. H9 was reproduced in the same kind of workspace, not on an independent machine.

11 Conclusion

Forecasting seasonal demand did not beat reactive cloud auto-tiering in a test whose rules were fixed before the data was seen. Managed tiering beat every forecasting rule we applied to all files, because it moves data between its own tiers without the fees a do-it-yourself plan pays. The saving that remains is real and large, 39% to 54% on two of three datasets. It comes mainly from leaving a few large, rarely read files in deep archive, and its price is that their readers wait up to two days. A year-ahead choice cannot keep that waiting at the rival's level every year, and a sudden change in demand removes the saving altogether. The open question is whether a controller that budgets waiting within the year could keep the saving without the risk. That needs new data and a new pre-registered test.

Artifact availability

Code, protocols, the decision log, per-run outputs and this paper's figures are at `https://github.com/summitkhatiwada22/seasonal-tiering-poc` [10]. Code is released under the MIT licence, and the paper and documentation under CC BY 4.0. The sealed test runs from the tag `freeze-g3`:

```
git checkout freeze-g3
uv run python -m sat.experiments.g3 run -workers 2
uv run python -m sat.experiments.g3 score
uv run python -m sat.experiments.g3 reproduce -workers 2
```

The raw data are public and are not redistributed; each downloaded file is recorded with its SHA-256 in the repository. `docs/RESULTS.md` lists every number in this paper with its source file. Figures are drawn by `scripts/paper/figures.py`.

Use of AI

The research question is the author's. So are the decision to pre-register, every gate decision (including the override of R1), and the interpretation. The code, analyses, figures and drafts of this text were produced with the help of Claude (Anthropic), an AI model, working under the author's direction. Every decision is logged in the repository's `docs/DECISIONS.md`, and the full history of the work is in its git log. The author takes responsibility for the content.

References

[1] Amazon Web Services. How S3 intelligent-tiering works. https://docs.aws.amazon.com/AmazonS3/latest/userguide/intelligent-tiering-overview.html, 2026. Read 2026-10-30.

[2] Amazon Web Services. Amazon S3 pricing (us-east-1). https://aws.amazon.com/s3/pricing/, 2026. Price data of 2026-10-15, read 2026-10-30.

[3] Amazon Web Services. Understanding archive retrieval options. https://docs.aws.amazon.com/AmazonS3/latest/userguide/restoring-objects-retrieval-options.html, 2026. Read 2026-10-30.

[4] S. Balakrishnan, R. Black, A. Donnelly, P. England, A. Glass, D. Harper, S. Legtchenko, A. Ogus, E. Peterson, and A. Rowstron. Pelican: A building block for exascale cold data storage. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014.

[5] A. N. Elmachtoub and P. Grigas. Smart "predict, then optimize". *Management Science*, 68(1):9–26, 2022. doi: 10.1287/mnsc.2020.3922.

[6] A. Erradi and Y. Mansouri. Online cost optimization algorithms for tiered cloud storage services. *Journal of Systems and Software*, 160, 2020.

[7] Google Cloud. Autoclass. https://docs.cloud.google.com/storage/docs/autoclass, 2026. Read 2026-10-30.

[8] T. Hoefler and R. Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015. doi: 10.1145/2807591.2807644.

[9] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021. URL https://otexts.com/fpp3/.

[10] S. Khatiwada. seasonal-tiering-poc: Code, protocols and results. https://github.com/summitkhatiwada22/seasonal-tiering-poc, 2026. Freeze tag `freeze-g3` (commit cc4b852).

[11] S. Liu, X. Mo, M. Hershcovitch, H. Zhang, A. Cheng, G. Girmonsky, G. Vernik, M. Factor, T. Bang, S. Ponnapalli, N. Crooks, J. E. Gonzalez, D. Harnik, and I. Stoica. SkyStore: Cost-optimized object storage across regions and clouds. *Proceedings of the VLDB Endowment*, 18(7):2084–2096, 2025. doi: 10.14778/3734839.3734846.

[12] T. Lykouris and S. Vassilvitskii. Competitive caching with machine learned advice. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, 2018.

[13] Microsoft. Smart tier for Azure Blob Storage. https://learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-smart, 2026. Read 2026-10-30.

[14] K. Mukherjee, R. Shah, S. K. Saini, K. Singh, Khushi, H. Kesarwani, K. Barnwal, and A. Chauhan. Towards optimizing storage costs on the cloud, 2023.

[15] S. Muralidhar, W. Lloyd, S. Roy, C. Hill, E. Lin, W. Liu, S. Pan, S. Shankar, V. Sivakumar, L. Tang, and S. Kumar. f4: Facebook's warm BLOB storage system. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014.

[16] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[17] npm, Inc. npm download counts API. https://api.npmjs.org/downloads/, 2026. Accessed 2026-10-29.

[18] K. P. N. Puttaswamy, T. Nandagopal, and M. Kodialam. Frugal storage for cloud file systems. In *Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys)*, 2012. doi: 10.1145/2168836.2168845.

[19] Seattle Public Library. An update about access to library technology systems. https://blog.spl.org/2024/05/28/an-update-about-access-to-library-technology-systems/, 2024. Posted 28 May 2024, updated 4 September 2024.

[20] Seattle Public Library. Checkouts by title. https://catalog.data.gov/dataset/checkouts-by-title, 2026. City of Seattle open data, accessed 2026-10-29.

[21] SNIA IOTTA Repository. IBM object store traces. https://iotta.snia.org/traces/key-value/36305, 2026. Accessed 2026-10-29.

[22] Z. Song, D. S. Berger, K. Li, and W. Lloyd. Learning relaxed Belady for content distribution network caching. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.

[23] A. Wei and F. Zhang. Optimal robustness-consistency trade-offs for learning-augmented online algorithms. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[24] Wikimedia Foundation. Pageview complete dumps. https://dumps.wikimedia.org/other/pageview_complete/, 2026. Accessed 2026-10-29.

[25] Z. Wu, M. Butkiewicz, D. Perkins, E. Katz-Bassett, and H. V. Madhyastha. SPANStore: Cost-effective geo-replicated storage spanning multiple cloud services. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013. doi: 10.1145/2517349.2522730.

A Additional results

Table 4. Sealed lines under the reported-only equal allowance (ours may use the full 1%).

Dataset	Years	Gap	Reads waited (rival / ours)	Wait hours (rival / ours)	Longest wait
Wikipedia	2023, 2024	+54.3%	0.245% / 0.277%	19,740 / 72,554	12 / 48 h
Wikipedia	2024, 2025	+52.7%	0.260% / 0.242%	20,731 / 64,373	12 / 48 h
npm	2023, 2024	+20.9%	0.089% / 0.447%	381 / 2,916	12 / 48 h
npm	2024, 2025	+49.3%	0.056% / 0.082%	269 / 2,112	12 / 48 h
Library	2023, 2024	−0.6%	0.351% / 0.629%	11,063 / 32,316	12 / 48 h
Library	2024, 2025	−5.3%	0.327% / 0.923%	10,236 / 47,592	12 / 48 h

Table 5. Sealed lines at L1 under the matched rule: costs (USD per year for the sample) and waiting.

Dataset	Years	Rival	Ours	Gap (95% interval)	Waited (rival / ours)	Hours (rival / ours)
Wikipedia	2023, 2024	35.874	16.380	+54.3% (51.0, 57.5)	0.245% / 0.277%	19,740 / 72,554
Wikipedia	2024, 2025	35.793	17.028	+52.4% (48.6, 56.0)	0.260% / 0.221%	20,731 / 59,755
npm	2023, 2024	0.624	0.381	+39.0% (9.1, 58.8)	0.089% / 0.118%	381 / 1,632
npm	2024, 2025	0.744	0.440	+40.9% (20.3, 59.0)	0.056% / 0.036%	269 / 972
Library	2023, 2024	12.530	14.490	−15.6% (−19.9, −12.0)	0.351% / 0.174%	11,063 / 5,026
Library	2024, 2025	11.962	13.456	−12.5% (−16.8, −9.0)	0.327% / 0.197%	10,236 / 5,850